NECTELIS — Reusable Proof-of-Capability Systems

Complete implementation blueprint for ClientFlow AI and SyncGuard

Document purpose: Build two real, repeatable technical demonstrations that can support many Upwork proposals without rebuilding a custom project for every listing.
Audience: NECTELIS agency founders, technical leads, implementation team, and future collaborators.
Status: Build specification and acceptance criteria; **not** a claim that the systems described here have already been implemented or tested.
Prepared: 14 October 2026.
Primary priority: Finish a small, verified ClientFlow AI demo quickly for current GoHighLevel automation opportunities, then finish both reusable proof packages.

> **North star:** Two working technical proofs, multiple honest presentations. We sell demonstrated engineering behavior—reliable integrations, safe data handling, repeatable deployment, and verified edge cases—not screenshots of impressive-looking but untested workflows.

---

1. Agency context and why these two proofs exist

1.1 Agency profile

NECTELIS is a multi-member technical agency offering **AI Apps & Integration, Web Development, AI & Machine Learning, and ERP/CRM Software**. Relevant services and skills include GoHighLevel (GHL), n8n, Make, Zapier, Vapi/Twilio voice-agent workflows, Microsoft 365/Graph integrations, Python/FastAPI, Node.js, REST APIs, webhooks, CRM systems, databases, and practical process automation.

Important positioning rule: The agency's stated skill areas define the *types of work it can consider*. Existing portfolio projects demonstrate selected experience, not the full limit of its technical knowledge. However, evidence must always distinguish **knowledge**, **demonstrated implementation**, and **verified production deployment**. Never turn a demo into a fictional client engagement.

1.2 The commercial problem

Upwork automation listings frequently require similar engineering patterns despite naming different platforms: capture incoming data, check existing records, apply business rules, update the CRM, book appointments, notify a team, synchronize a second system, prevent duplicates, and recover from failures. Building a new proof for every listing is wasteful. Generic slides and toy "form → email" demos are also weak proof.

Our answer: Build a reusable mini-portfolio with two independently demoable systems, each configurable by industry and connector. Use one shared evidence library with short video clips, screenshots, test results, and written case studies.

1.3 Proofs and target jobs

Proof	Core business result	Particularly useful for
ClientFlow AI	Capture and qualify an inquiry, create/update the correct CRM record, handle a booking, update the pipeline and notify the team	GHL automation, AI receptionist, lead management, appointment setter, CRM workflows, SaaS/API integrations, agency client work

Proof	Core business result	Particularly useful for
SyncGuard	Synchronize changing records safely between two systems, prevent duplicate writes, recover from errors, and show auditable results	n8n troubleshooting, REST/webhook integrations, Microsoft Graph/SharePoint/Outlook, contact sync, data migration, Python/Node backend, operational automation

Out of scope: These two demos are not universal proof of advanced ML training, regulated healthcare compliance, specialized ERP software, deep front-end design, PostGIS administration, or every third-party platform. They support those applications only where an honestly relevant component overlaps.

1.4 Success criteria across both proofs

1. An evaluator can run a demo using safe synthetic data without access to any real client's secrets.
2. Each proof completes a real multi-step workflow **and** clearly demonstrates at least three failure/edge cases.
3. It is possible to change the use case through documented configuration and adapters without rewriting the full orchestration.
4. A clean test report distinguishes **passed**, **failed**, **not implemented**, and **requires live integration**.
5. Public artifacts contain no credentials, customer data, private n8n/CRM endpoints or confidential workflows.
6. A 60–120-second proof video and three short topic-specific clips can be reused across many proposals.
7. The current GHL job can be supported by **one accurately labeled demonstration**; no unsupported "native GHL" claims.

2. Global design principles and engineering decisions

2.1 Build lean, not theatrical

- **Functional reliability over UI:** a simple request form and clean execution trace beat an elaborate dashboard that hides failures.
- **Adapter-based reuse:** isolate GHL/other CRM actions from lead-processing logic; isolate Microsoft Graph/other APIs from synchronization logic.
- **Configurable rules:** company name, industry, time zone, custom fields, lead statuses, pipeline stages, notifications, calendar and connector details must not be hardcoded inside node logic.
- **Deterministic core:** do not use an LLM to decide whether a record already exists, whether a time slot is available, or whether a payment succeeded. AI qualification is optional and its output is validated.
- **One source of truth per field:** the destination cannot silently override authoritative source values.
- **Privacy-first samples:** synthetic people, emails, phone numbers, appointments and records only. No unnecessary live outreach.
- **Minimal scope with visible extension points:** design voice, social DM, native GHL snapshots, OAuth and multi-tenancy as optional modules, not false promises in the core.

2.2 Recommended implementation stack

Layer	Preferred choice	Practical substitute
Orchestration	Existing n8n Cloud workspace, or self-hosted n8n	Make/Zapier only if a specific demo needs it
API and adapter layer	Python 3.11+ with FastAPI and Pydantic	Node.js/Express if a team member prefers it; pick **one** primary backend per proof

Layer	Preferred choice	Practical substitute
Persistence	PostgreSQL for reliable state/audit	n8n Data Tables for very small proof, with limitations documented; SQLite for local single-user demo
CRM	Authorized GoHighLevel sub-account/API	Clearly labeled **mock CRM adapter** until legitimate GHL test access exists
Calendar	Authorized GHL calendar or Google Calendar test account	Mock availability/booking service, clearly labeled
Notifications	Test mailbox, internal webhook, or n8n test channel	Local log records only in offline mode
Voice	Existing authorized Vapi/Twilio demo, optional	Simulated transcript/inbound event; **never label it as live voice**
Dev environment	Git/GitHub, .env.example, Docker Compose when available	Local Python + hosted n8n when Docker/WSL is unavailable
Test runner	pytest + API test fixtures	Node equivalent if backend is Node

Cost policy: Core demos should work without paid inference, messaging, telephony or production CRM subscriptions. Real GHL/voice/SMS integrations are additional demonstration modes when authorized access and pricing are available. Live telephone numbers, SMS/A2P, calling minutes, messaging, platform seats, third-party maps, and paid LLM APIs are **not** assumed free.

2.3 Required execution modes

- `DEMO_MODE=mock`: entire scenario runs using synthetic events, mock CRM/calendar/APIs; suitable for offline sharing. Clearly labeled.
- `DEMO_MODE=live_test`: authorized test credentials connect to real GHL/calendar/provider accounts; all recipients are allow-listed.
- `DRY_RUN=true`: build the plan and audit result without writing to downstream services.
- `LIVE_OUTBOUND_ENABLED=false` by default: prevents accidental real messages, calls, notifications or mass outreach.
- Environment-specific secrets loaded from an ignored `.env`, n8n credentials or approved secret store; never commit tokens.

3. Proof A — ClientFlow AI

3.1 Identity and purpose

Name: ClientFlow AI — Reusable Lead-to-CRM and Appointment Automation.
One-sentence pitch: “A configurable inquiry-to-booking engine that safely qualifies leads, upserts CRM contacts, updates opportunities, schedules appointments and reports failures.”
Business problem: Businesses lose leads when inquiries arrive across multiple channels, records duplicate, follow-ups fail, appointments conflict, and staff lack reliable visibility. Agencies need a deployment pattern adaptable to many customer accounts.

Primary demo industry: Home services (safe, easy to explain).
Alternative profiles: Real estate, consulting/coaching, local agencies, retail services. Healthcare scenarios remain illustrative only unless privacy, compliance and scheduling safeguards are reviewed for the actual jurisdiction.

Intended viewers: Agency owners, GHL specialists, CRM managers, AI-voice buyers, API integration clients, and hiring managers looking for operational reliability.

3.2 Scope: core versus extensions

Must-have working core

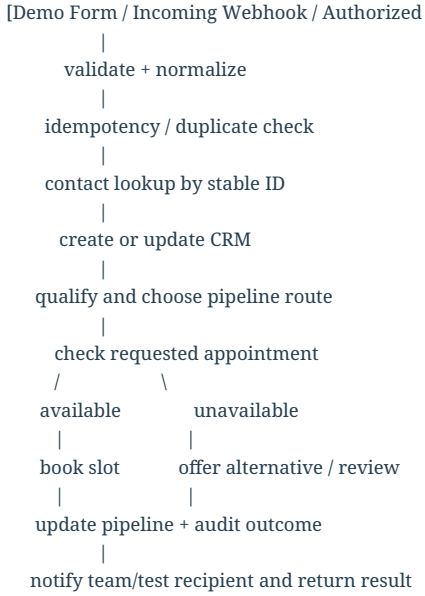
- Website/demo inquiry form or inbound webhook.
- Input schema validation with helpful error handling.
- Canonical lead/contact identification and existing-contact lookup.
- CRM **upsert** through a swappable connector (real GHL only if test access exists).
- Configurable qualification and routing rules.
- Pipeline/opportunity stage update.
- Calendar availability check and **one** verified booking or a no-slot alternative.
- Consent-aware internal/lead notifications in test-only mode.
- Duplicate-event prevention, retries where safe, error audit and manual retry mechanism.
- At least one completed end-to-end **success test** and several negative tests.
- README, setup instructions, configuration file, test evidence and video.

Optional extensions, only after the core passes

- Real Vapi/other voice inbound capture and natural-language intake.
- AI qualification through a paid/free LLM model, constrained to a validated JSON output.
- Native GHL workflow and snapshot import/export, calendar/pipeline configuration and client install procedure.
- Missed-call text-back; consent and quiet hours before any real SMS.
- No-show rebooking, 2-way conversational SMS, social DM, multi-tenant client management.
- Live dashboard/analytics and human-in-the-loop approvals.

Boundary: A GHL **API-connected** proof is not the same as a demonstration of **native GHL workflows or snapshots**. Claim and label them separately. Likewise a fake webhook pretending to be a phone call is not a voice-agent demo.

3.3 Architecture and end-to-end scenario



Suggested n8n workflow node labels (clear enough for a nontechnical manager): **Receive Lead → Validate Lead → Check Duplicate Event → Find CRM Contact → Create or Update Contact → Qualify Lead → Check Calendar → Book or Request New Time → Update Pipeline → Notify Team → Save Audit Result**. Keep failure branches visible, not hidden inside one opaque script.

3.4 Sample input/output contract

Inbound event (lead.received)


```
{
 "event_id": "demo-event-1001",
 "source": "website_form",
 "received_at": "2026-10-14T11:00:00Z",
 "client_key": "home-services-demo",
 "lead": {
  "first_name": "Alex",
  "email": "alex.demo@example.com",
  "phone": "+15550101001",
  "service": "air_conditioner_repair",
  "postal_area": "Demo District",
  "message": "Need service this week",
  "consent_to_contact": true
 },
 "requested_slot": "2026-10-16T14:00:00-04:00"
}
```


Success result (lead.processed)


```
{
 "event_id": "demo-event-1001",
 "status": "booked",
 "crm_action": "created",
 "contact_id": "contact_demo_104",
 "opportunity_stage": "Appointment Booked",
 "appointment_id": "appt_demo_209",
 "notification_status": "test_delivered",
 "audit_id": "audit_demo_301"
}
```


Data-quality rules: Require either a valid, permitted contact method or an explicit review path. Normalize emails cautiously, phone numbers to E.164 where feasible, dates with an IANA timezone, and consent as a first-class field. Never use email or phone alone as a guaranteed global identity when the downstream system has stronger IDs. Mask sensitive values in logs.

3.5 Configurable client profile (no code edits for ordinary adaptation)


```
client_key: home-services-demo
company_name: Demo Home Services
industry: home_services
timezone: America/New_York
crm_adapter: mock_ghl          # or ghl_test after authorized setup
calendar_adapter: mock_calendar   # or ghl_test/google_test
pipeline:
 new: New Inquiry
 qualified: Qualified
 booked: Appointment Booked
 review: Needs Human Review
qualification:
 required_fields: [service, consent_to_contact]
 supported_services: [air_conditioner_repair, plumbing, electrical]
notifications:
 internal_enabled: true
 external_enabled: false
 test_recipient_allowlist: [team.demo@example.com]
reliability:
 max_transient_attempts: 3
 idempotency_window_hours: 72
```


Adaptation rule: A real customer deployment also needs actual stage IDs, API authorization, consent policy, calendar access, allowed calling windows, locale, support ownership, and explicit testing. Changing a YAML company name is **not** sufficient to deploy regulated or complex verticals.

3.6 Required functional requirements

ID	Requirement	Evidence needed
CF-01	Accept valid inbound lead events	Request and accepted response
CF-02	Reject invalid payloads without creating records	4xx result and audit reason
CF-03	Normalize and validate identifying fields	Normalized test output
CF-04	Detect/reject replayed event_id	Second identical run produces no new contact/booking
CF-05	Find existing contact; create new only if absent	Create and update demonstrations
CF-06	Apply configurable lead qualification rules	Qualified and manual-review examples
CF-07	Set correct stage/owner using configured IDs	Visible CRM/mock stage result
CF-08	Check booking availability immediately before create	Available/unavailable test
CF-09	Prevent double booking or duplicate downstream booking actions	Repeated event test and conflict response
CF-10	Trigger only authorized test communications	Allow-list evidence; no customer messages
CF-11	Handle 429/5xx temporary API errors safely	Retry evidence and bounded attempts
CF-12	Isolate 4xx validation/auth failures for review	Logged failure, no blind retry loop
CF-13	Show a complete per-event audit trail	Correlated event/audit IDs
CF-14	Reconfigure demo for another industry with profile change	One second client config, successful smoke test
CF-15	Clearly label mock vs real connector mode	README + video captions

3.7 Reliability and security requirements

1. **Idempotency:** unique (client_key, source, event_id) processing ledger with status; do not rely on workflow execution count alone. Where downstream API supports idempotency keys, pass one. If booking API lacks atomic holds/idempotency, acknowledge remaining race conditions and use a serialized/locked booking step.
2. **Contact identity:** lookup by stable CRM ID when known, otherwise normalized email/phone according to connector rules; avoid duplicate contact creation on retried webhooks.
3. **Retries:** for 429 honor Retry-After; retry 5xx/timeouts with bounded exponential backoff plus jitter; distinguish “request timed out after server possibly created record” and reconcile before retrying a write.
4. **Error isolation:** 400/422 bad input → manual correction; persistent 401/403 → stop and alert; repeated provider outages → retry queue and clear final status.
5. **Access:** verify signed webhooks where supported; apply API tokens with minimum permissions; do not expose secret-bearing webhook URLs or PII publicly.

6. **Communications:** test-only outbound by default; no unsolicited messages; account for opt-in, business hours, local SMS/WhatsApp platform rules and A2P where relevant.
7. **Audit:** log event ID, timestamps, stage transitions, safe request metadata, destination record IDs, attempts, and the final outcome. Never log raw passwords or tokens.
8. **Data minimization:** purge sample data on demand and keep only what the demo requires.

3.8 ClientFlow AI acceptance tests

Test	Scenario	Expected result
CF-T01	New permitted lead + available slot	Contact created, stage booked, one appointment, team notified
CF-T02	Existing lead changes phone/service	Existing contact updated, no duplicate contact
CF-T03	Replay exact same webhook	No duplicate contact, appointment, or notification
CF-T04	Missing required service	Human review or validation response, no silent booking
CF-T05	Missing consent for outbound message	No outbound message; internal handling remains clear
CF-T06	Requested slot unavailable	No booking, alternative/review recorded
CF-T07	Two rapid requests for one slot	At most one booking or explicit documented race limitation in mock
CF-T08	CRM returns 429 then 200	Bounded retry; exactly one intended final update
CF-T09	CRM returns 401/403	Stop connector writes and alert; no endless retry
CF-T10	Calendar API times out	Reconcile booking state before retry; no duplicate
CF-T11	Downstream team notification fails	Lead/booking result preserved; notification failure logged
CF-T12	Switch home services → real estate config	Same engine processes another demo profile
CF-T13	DRY_RUN is enabled	No downstream writes, clear planned actions
CF-T14	Non-allowlisted recipient in live-test mode	Send blocked and auditable

Minimum publish gate: CF-T01 through CF-T06, CF-T08, CF-T09, CF-T11, CF-T13 and CF-T14 pass. CF-T07/CF-T10 must be tested or clearly marked as not yet assured; never describe booking as race-proof without evidence.

3.9 Build procedure — ClientFlow AI

1. **Set up repository and environment:** create FastAPI app, `requirements.txt`/lockfile, `.env.example`, demo profile, sample payloads, database schema, basic pytest suite. Include a README stating which connectors are mocked.
2. **Build adapters and audit:** define `CRMAdapter` and `CalendarAdapter` interfaces, mock implementations, event ledger and audit records. Test mocks independently.

3. **Implement lead intake:** create /api/demo/lead endpoint or n8n webhook, schema validation, normalization and signature checks where real webhook provider supports them.
4. **Implement stable CRM upsert:** lookup → compare → update/create; return source record and destination IDs. Test duplicate/replay paths.
5. **Add qualification and pipeline mapping:** deterministic rules first; optional validated AI classification later.
6. **Add booking:** check available slots and confirm booking, no-slot branch, idempotent execution and human escalation on ambiguous outcomes.
7. **Add notifications and error branches:** internal test notification, dead-letter/review state, audit timeline and bounded retry.
8. **Wire n8n:** keep nodes descriptive; n8n should coordinate, not hide all business rules inside unreviewable Code nodes.
9. **Run test matrix:** store sanitized test output, screenshots and a pass/fail report. Do not skip negative tests.
10. **Connect authorized GHL test account (if available):** map native custom field IDs, contact/opportunity IDs and calendars, test the real connector separately. If access unavailable, publish the mock proof honestly and do not claim native GHL experience from it.
11. **Create artifacts:** screenshots of the workflow, contact result, successful booking, duplicate-run evidence and failure/audit trace; record videos.

3.10 ClientFlow optional native GHL validation

When a GHL test sub-account is available, record whether these are genuinely tested: contacts lookup/create/update; workflows and triggers; pipeline/opportunities; calendar availability and booking; custom values and custom-field IDs; user/team notification; snapshot export/import behavior; phone-number/A2P dependencies; external API authorization. Capture successful *and failed* results. Some features may require account permissions, plan entitlements, region/platform approval or additional cost; do not assume universal availability.

4. Proof B — SyncGuard

4.1 Identity and purpose

Name: SyncGuard — Reliable API Data Synchronization and Recovery Framework.
One-sentence pitch: "A verifiable, change-aware synchronization pipeline that sends only new or modified records, respects API limits, records every outcome, and retries safely."
Business problem: Companies synchronize contacts, invoices, CRM records and staff data across systems; naïve workflows repeatedly create duplicates, overwrite newer values, fail under rate limits and provide no clear repair path.

Primary demo: One-way contact sync between **Source API** and **Destination CRM** using synthetic records.
Useful adapters later: SharePoint → Outlook/Graph, GHL ↔ another CRM, Airtable ↔ database, ERP → reporting store, legacy REST API → CRM.

Why not begin with two-way sync? Two-way sync doubles conflict, deletion and ownership complexity. Build and prove one-way first, with one authoritative source and explicitly defined update semantics.

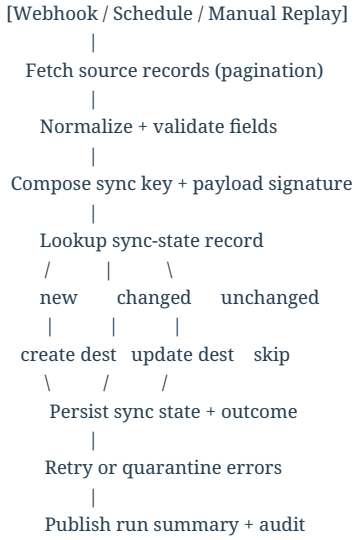
4.2 Must-have working core

- Source API with records, IDs, updates, pagination and a cursor/watermark or repeatable full scan.
- Normalization and schema validation.
- Destination lookup by durable source-to-destination mapping.
- Per-field change detection via canonical payload signature (hash).

- CREATE for new records, UPDATE for changed records, SKIP for unchanged records.
- Per-record synchronization state and auditable run summaries.
- Rate-limit aware bounded retries and clear distinction between recoverable/nonrecoverable errors.
- Safe replay and restart after partial failure.
- Dry-run and allow-listed test mode.
- Real API adapter only when authorized; mock provider with configurable error injection always available.
- Minimal setup instructions, fixtures, tests, demo videos and example output.

Optional extensions: Graph test-tenant connector, GHL connector, scheduled polling, webhook-triggered sync, deletion policy, reconciliation report, selective backfill, two-way sync with an explicit conflict-resolution policy, small admin UI.

4.3 Architecture and synchronization flow



Suggested n8n node labels: **Start Sync → Fetch Source Page → Normalize Record → Look Up Sync State → Detect Changes → Create or Update Destination → Respect Rate Limits → Save Sync Result → Send Run Summary**. Put error/recovery branches in view.

4.4 Fundamental concepts (important for implementation)

- **Source record ID:** a stable ID from System A; not an email address that can change.
- **Destination record ID:** the ID returned by System B after creating or locating the record.
- **Sync key:** identifies **which source record is mapped to which destination/account**, e.g. `source_system + source_record_id + destination_system + destination_account_id`. For a contact copied into eight employees' Outlook mailboxes, there are eight separate mappings.
- **Normalized payload:** only the fields we intentionally synchronize, with consistent null/phone/string formatting. Do not hash volatile metadata such as request time.
- **Payload signature:** SHA-256 of a deterministic canonical serialization of the normalized synchronized fields. Equal signatures normally mean no meaningful mapped-field change.
- **Last successful state:** update the stored signature **only after the destination confirms success**. Otherwise a failed write could be incorrectly treated as already synchronized.
- **Idempotency:** repeated ingestion should converge to one intended destination state, using mapping and reconciliation rather than blindly creating another record.
- **Watermark/cursor:** progress through the source dataset; move the checkpoint only when the processing policy is safe for partial failures.

4.5 Example normalized record and sync state


```
{
 "source_system": "mock_sharepoint",
 "source_record_id": "25",
 "destination_system": "mock_outlook",
 "destination_account_id": "recipient_demo_A",
 "data": {
  "given_name": "Taylor",
  "family_name": "Morgan",
  "email": "taylor.morgan@example.com",
  "phone": "+15550101555",
  "company": "Demo Construction Co"
 }
}
{
 "sync_key": "mock_sharepoint:25:mock_outlook:recipient_demo_A",
 "destination_record_id": "outlook_demo_302",
 "last_successful_signature": "sha256:example-not-a-real-hash",
 "last_synced_at": "2026-10-14T11:30:00Z",
 "last_result": "updated",
 "last_error": null
}
```


4.6 Minimal persistent data model

Table	Key fields	Purpose
sync_connections	connection ID, source/destination type, account, status	Nonsecret connection metadata
sync_mappings	composite sync key, destination record ID, last successful signature	Record-to-record identity and change detection
sync_runs	run ID, started/finished, cursor, status, counts	Run-level checkpoints and summary
sync_attempts	run ID, sync key, attempt, HTTP status, safe error, next retry	Recovery and diagnostics
sync_dead_letters	sync key, source payload reference, reason, remediation status	Nonretryable or exhausted failures for manual handling

Store secrets separately. Avoid unnecessary copies of personal data in audit tables. Use unique database constraints on composite sync keys; a purely in-memory “seen items” set is insufficient for repeat runs.

4.7 Error handling matrix

Event	Expected handling	Never do this
Source 429	Honor Retry-After, slow down concurrency	Hammer provider with immediate parallel retries
Source/destination 5xx	Retry with backoff and maximum attempts	Retry forever without alerting
Request timeout after write	Re-query mapping/remote item before another write	Assume failure means create never happened
400/422 invalid payload	Log validation issue, quarantine item	Retry identical invalid input repeatedly
401/403 authorization problem	Stop affected connection; notify owner	Mask permissions failure as a transient outage
Destination 404 on mapped record	Apply documented missing-remote policy: recreate, relink or quarantine	Silently create a replacement without policy

Event	Expected handling	Never do this
409 conflict	Re-fetch and resolve according to source-of-truth policy	Overwrite an unknown newer change without review
Partial run crash	Resume or safely replay outstanding items	Mark entire batch completed regardless of failures
Deleted source record	Retain, archive or delete per explicit client policy	Automatically delete target records without approval

Graph-specific caution: Microsoft Graph permission modes, mailbox access and per-user licensing depend on the tenant and endpoint; require authorized test tenant and least-privilege access. Never claim the demo has enterprise permissions unless tested.

4.8 Required functional requirements

ID	Requirement	Evidence needed
SG-01	Fetch paginated source dataset	Multiple-page trace, no dropped records
SG-02	Validate and normalize fields	Raw→normalized example
SG-03	Generate stable composite sync key	Same record/account retains same mapping
SG-04	Generate stable signature of mapped data	Same input same hash; meaningful change new hash
SG-05	New item creates one destination record	Create result + mapping
SG-06	Changed item updates original destination ID	Update result + same ID
SG-07	Unchanged item causes zero destination writes	Clear skip evidence
SG-08	Same source to two accounts stays isolated	Distinct per-account mappings
SG-09	Respect rate limit and resume safely	429 injection + delayed retry trace
SG-10	Handle permanent failure without data loss	Dead-letter entry and run status
SG-11	Recover after restart/partial batch	Re-run converges without duplicates
SG-12	Output understandable summary	Created/updated/skipped/failed counts
SG-13	Safe DRY_RUN and replay	No writes in dry-run; safe rerun
SG-14	Change connector/profile without rewriting sync algorithm	Second adapter smoke test

4.9 SyncGuard acceptance tests

Test	Scenario	Expected result
SG-T01	Five new contacts	Five creates, five mappings
SG-T02	Run again unchanged	Zero writes; five skips
SG-T03	Change one phone number	One update; four skips; no creates

Test	Scenario	Expected result
SG-T04	One contact copied into two destination accounts	Two distinct destination mappings
SG-T05	Simulate 429 then success	Respect retry schedule, eventual success once
SG-T06	Simulate 400 invalid email	One quarantined item; others continue
SG-T07	Simulate persistent 401	Affected connection stops with alert
SG-T08	Kill process mid-batch, then rerun	Convergent state without duplicate remote records
SG-T09	Simulate timeout after destination create	Reconcile; no second remote contact
SG-T10	Destination record manually deleted	Explicit policy path, no silent surprise
SG-T11	Source has duplicate event/page overlap	No duplicate destination writes
SG-T12	DRY_RUN enabled	Correct change plan, zero destination writes
SG-T13	Cursor/pagination boundary	All source records processed exactly as intended
SG-T14	Run summary	Counts, failures and durations reconcile with audit

Minimum publish gate: SG-T01–SG-T07, SG-T11, SG-T12, SG-T13 and SG-T14 pass, plus a documented restart/reconciliation result before claiming robust failure recovery. Never claim “exactly once delivery” across arbitrary remote APIs; aim for safe, idempotent or reconciled outcomes.

4.10 Build procedure — SyncGuard

1. Create two small provider simulators: source list/update API with pagination and destination create/search/update API with configurable error injection.
2. Define source/destination adapter contracts and canonical field-mapping configuration.
3. Implement deterministic normalizer and payload-signature function with unit tests.
4. Add PostgreSQL mapping and run/attempt tables with uniqueness constraints.
5. Build the initial **one-way** sync flow: fetch → normalize → compare → create/update/skip → commit successful state.
6. Implement bounded concurrency, 429/backoff behavior and retries only where safe.
7. Add partial-failure replay, dead-letter list and per-run summary.
8. Run the full fixture matrix; collect create/update/skip comparison and failure traces.
9. If a legitimate tenant is available, add an optional SharePoint/Graph or GHL test connector and record an additional real API trace.
10. Package configuration profiles, tests, sanitized screenshots, videos and deployment guide.

5. Shared repository, modules and file deliverables

5.1 Recommended repository organization

```
nectelis-proof-systems/
  README.md                 # honest overview, status, demo modes
  LICENSE                   # choose after checking ownership/reuse rights
  .gitignore
```

```
.env.example                # placeholders only; never real keys
docs/
  architecture.md
  proof-matrix.md           # job need → evidence asset
  recording-guide.md
  test-results.md
  privacy-and-security.md
  client-handoff-template.md
  demo-data-policy.md
clientflow-ai/
  README.md
  config/
    home-services-demo.yaml
    real-estate-demo.yaml
  api/                      # FastAPI or chosen single backend
  adapters/                 # mock CRM, optional GHL; calendar
  workflows/                # sanitized n8n exports
  fixtures/                 # synthetic requests/responses
  tests/
  screenshots/
  scripts/
syncguard/
  README.md
  config/
    contacts-one-way.yaml
    api-to-crm-demo.yaml
  api/
  adapters/                 # mock source, destination, optional Graph/GHL
  workflows/
  migrations/               # persistent state schema
  fixtures/
  tests/
  screenshots/
  scripts/
portfolio/
  clientflow-case-study.md
  syncguard-case-study.md
  demo-video-links.md
  proposal-proof-snippets.md
```

Avoid exposing confidential artifacts: Do not copy customer-owned workflow exports, actual contact lists, account tokens or proprietary screenshots into a public portfolio. Recreate the same general technique independently if permission is absent. Protect webhook endpoints, and use neutral test domains and dummy data.

5.2 The exact required assets per proof

Asset	Format	Non-negotiable content
Setup guide	README.md	Requirements, install/run, demo modes, environment, limitations, cleanup
Diagram	PNG/SVG or Markdown	Trigger, adapters, decision points, successful/failure branches
Workflow	Sanitized n8n export JSON	Importable, named nodes, no credentials
Sample config	YAML	At least two industry/connector profiles
Demo fixtures	JSON/CSV	No client/user data; success and failure cases
Automated tests	pytest or equivalent	Deterministic assertions and clear results

Asset	Format	Non-negotiable content
Test report	Markdown/PDF	Case IDs, actual outputs, pass/fail, run date, connector mode
Screenshots	PNG	Workflow, success record, negative test, audit summary
Video master	90–120 seconds	Actual running system and truthful connector labels
Short clips	20–40 seconds each	CRM/booking, API sync/change detection, failure/retry
Case study	1 page each	Problem → design → implemented behavior → verified results → limitations
Config guide	Markdown	What can be changed, what needs client credentials/consent

For Upwork applications, use the **most relevant 1–2 assets**, not the whole zip. A single link to a clean overview plus a relevant clip is enough. Different jobs need different angles, not different builds.

6. Recording plan — reusable proof library

6.1 Master ClientFlow AI video (90–120 seconds)

Time	Show on screen	Suggested narration
0:00–0:12	Workflow overview	“This is a configurable lead-to-booking automation.”
0:12–0:30	Submit synthetic lead	“A new inquiry enters through a webhook and is validated.”
0:30–0:48	CRM lookup/upsert	“The system finds or creates the right contact instead of blindly adding duplicates.”
0:48–1:06	Qualification, stage, booking	“Rules route the lead and confirm availability before a booking is recorded.”
1:06–1:25	Replay/failure test	“A repeated event doesn’t create a second booking; failures are logged.”
1:25–1:45	Config + test output	“Industry settings are configurable; these are the passing checks and current limitations.”

If a part is not built, remove that line rather than narrate an imaginary capability. Display **MOCK CRM DEMO** or **LIVE GHL TEST ACCOUNT** visibly according to the true mode.

6.2 Master SyncGuard video (90–120 seconds)

Time	Show on screen	Suggested narration
0:00–0:15	Systems and mapping	“Source records sync to an API destination with persistent mapping state.”

Time	Show on screen	Suggested narration
0:15–0:32	First run	“Five new records result in five creates.”
0:32–0:48	Second run	“No changed data: five skips, zero extra writes.”
0:48–1:05	Change one field	“One phone change triggers one update against the original destination ID.”
1:05–1:28	Inject 429 or malformed record	“The pipeline backs off on rate limits and isolates unrecoverable errors.”
1:28–1:45	Run summary and config	“Each outcome is logged, and adapters can be replaced without rewriting the sync logic.”

6.3 Create four evergreen clips

1. **GHL/CRM clip:** contact upsert + pipeline + booking (prefer real test connector if legitimately available).
2. **AI voice clip:** actual phone/audio tool invocation only if a real authorized voice integration is demonstrable.
3. **API engineering clip:** normalized data, mapping, signature, create/update/skip.
4. **Reliability clip:** repeated event, 429 and safe recovery; easy to repurpose for troubleshooting jobs.

Recording quality: 1080p where possible; enlarge important text; hide credentials, browser account identifiers and personal inboxes; use narration or clean captions; show before/after outputs; no fabricated test logs, misleading claims or client names without permission. Host recordings on a stable view-only service; stay within Upwork’s precontract communication rules.

7. Reuse matrix: matching one proof to many proposals

Job type	Best proof	Lead with	What to swap before sending
GHL automation specialist	ClientFlow	CRM upsert, triggers, pipeline and testing	GHL-specific clip + client workflow terminology
AI receptionist / AI appointment setter	ClientFlow + actual voice clip if available	Voice-to-action, booking, fallbacks	Channel/platform statement, booking use case
n8n specialist	Both	Readable nodes, conditional paths, operations	n8n screenshots, workflow JSON preview
API/webhook troubleshooter	SyncGuard	Error classification, change detection, retries	Failure clip and API architecture note
SharePoint/Outlook/Graph sync	SyncGuard	Mapping per recipient, update-not-create, throttling	Authorized Microsoft test integration if available
CRM migration/data sync	SyncGuard	Source of truth, upsert, checkpoints and reconciliation	Data mapping example and migration assumptions
Backend Python/Node API	Both	Validation, adapter APIs, data model and automated tests	GitHub tests and API contracts
Client agency multi-account builds	ClientFlow	Repeatable client config, safe handover, boundaries	Two different demo profile screenshots

Job type	Best proof	Lead with	What to swap before sending
Automation quality/maintenance	Both	Failure isolation, logs, idempotency	Before/after or error-injection case study

7.1 Ten-minute per-proposal reuse procedure

1. Identify the employer's **real operational problem**, not only their tool name.
2. Select **one** matching proof and **one** most relevant tested behavior.
3. Swap industry wording in the case-study introduction only when the underlying behavior supports it.
4. Include the most relevant 20–40-second clip or sanitized screenshot.
5. State the specific connector status: real GHL test, mock GHL, Microsoft test tenant, or simulated API.
6. Explain how the candidate would adapt the system to the employer's stack.
7. Identify the first bounded deliverable and one intelligent scoping question.
8. Remove claims that cannot be supported by actual execution or the selected person's work.

Reusable proof line: "We built a configurable [lead-handling / sync] demonstration that [observed behavior]. In a similar client system, we would adapt the connector and business rules, then validate [one client-specific risk]."

8. Execution plan, priority, and team ownership

8.1 Immediate fast track (protect current GHL application opportunity)

Target: 3–5 focused hours **if** you can reuse an existing demo environment and the basics are already configured; otherwise this is a timebox, not a promise.

- **Hour 1:** select an existing authorized GHL-related implementation or establish a clearly labeled mock; collect synthetic lead fixture and define the success path.
- **Hour 2:** run and verify contact upsert + pipeline or booking step; capture actual output.
- **Hour 3:** run one duplicate-event case and one failure case; save the evidence.
- **Hour 4:** record the 60–90-second proof and create a one-page case study; check privacy and claims.
- **Hour 5 (buffer):** fix reproducibility, upload sanitized assets, tailor the selected job's proposal and submit if the listing is still open.

Important: Do not delay a recently posted job several days waiting for a perfect generalized platform. If a truthful partial demo already exists, use it. If no viable live GHL environment exists, don't pretend a mock demonstrates native GHL workflows.

8.2 Full implementation sequence

Phase	Goal	Exit gate
P0 — Inventory & access	Decide live versus mock modes, owner assignments and repo	Confirm tools and secure credentials
P1 — ClientFlow core	End-to-end intake, CRM, booking, audit	Core successful and duplicate tests pass
P2 — ClientFlow resilience	Error scenarios, second profile, recorded evidence	Publish gate met
P3 — SyncGuard core	Paginated source, normalization, create/update/skip	T01–T04 pass

Phase	Goal	Exit gate
P4 — SyncGuard recovery	429/replay/error isolation and run summary	Publish gate met
P5 — Portfolio packaging	Case studies, videos, screenshots, README	Reviewer can reproduce in test mode
P6 — Optional live adapters	Genuine GHL/Graph/voice integrations with authorization	Connector-specific validation evidence

Indicative effort: lean verified demos can take a few focused working days if access and starter code exist; external access, telephony and advanced native GHL setup can extend this substantially. No fixed delivery claim before technical inventory.

8.3 Suggested responsibility split for a three-person agency

Role	Owns	Completion evidence
Automation owner	n8n flows, branches, readable workflow exports, demo execution	Workflow + run history
Backend/integration owner	FastAPI/Node adapters, schemas, idempotency, persistent state	Code + test results
QA/documentation owner	Negative tests, screenshots, repeatable instructions, portfolio packaging	Test matrix + video + safe release

These are **role assignments**, not assumptions about which individual knows which skills. Rotate as needed, but designate one accountable owner per deliverable.

9. Security, authenticity and client-ownership rules

- Use synthetic names, email addresses (example.com) and nonworking demonstration phone numbers.
- Use provider sandbox numbers/accounts or permitted test recipients; real SMS/voice incurs possible fees and platform restrictions.
- No client secrets, PII, webhook secrets, private URLs, tokens or customer contact exports in videos/repositories.
- Disable all automated outbound messages in offline/mock mode, and allow-list recipients in live-test mode.
- Validate request signatures, input size, schema and authorization whenever real endpoints are exposed.
- Maintain separate test and production credential stores; never expose production accounts to a public demo.
- Do not use confidential client code, screenshots or workflows without authorization and ownership clearance.
- Give honest status labels: **Built & Tested**, **Built but Not Tested**, **Mocked**, **Planned**, **Requires Paid/Client Access**.
- For agency proposals, identify what the agency/team built and what the submitting member personally implemented when requested. Avoid unverifiable testimonials, results or certifications.
- Do not present a demo as HIPAA/GDPR/CASL-compliant or enterprise-certified merely because it has a privacy statement. Actual deployment needs a specific legal/security review.

10. Final definition of done — release checklist

A proof is **ready to show to employers** only when every relevant checkbox below is satisfied. Unknowns must be labeled openly.

ClientFlow AI

- ☐ Success path works with synthetic data and known connector mode.
- ☐ Existing-contact update does not create a duplicate.
- ☐ Duplicate inbound event does not duplicate the booking.
- ☐ Unavailable slot or missing required input has a safe outcome.
- ☐ CRM/API 429 and persistent 401 failures have bounded documented behavior.
- ☐ Notifications and communications are test-only unless explicitly authorized.
- ☐ Audit trail and case-specific output can be shown on screen.
- ☐ At least two client configurations work without code rewrites.
- ☐ README, fixtures, test table, screenshot set and 90-second video exist.
- ☐ Real versus mock GHL/native workflow capabilities are stated accurately.

SyncGuard

- ☐ First run creates expected records; second unchanged run performs zero writes.
- ☐ One changed field updates exactly the mapped destination record.
- ☐ Multiple destination accounts have isolated mapping keys.
- ☐ Payload signatures are deterministic and exclude irrelevant volatile fields.
- ☐ Partial failure/retry state does not mark failed writes as successful.
- ☐ 429, permanent 400 and expired-auth cases are handled distinctly.
- ☐ Resume/replay and source pagination are tested or limitations stated.
- ☐ Summary counts reconcile with actual audit rows.
- ☐ README, fixtures, test table, screenshot set and 90-second video exist.
- ☐ Real versus mock Microsoft/GHL integrations are stated accurately.

Portfolio packaging

- ☐ No secrets or customer data in any publicly shared asset.
- ☐ Links work without requesting a hiring manager's login to a private client account.
- ☐ Video has visible before/after output and readable labels.
- ☐ Case studies describe only measured, reproducible behavior.
- ☐ One Upwork-focused proof clip is readily available for each of: GHL/CRM, voice (only if real), API sync, troubleshooting.
- ☐ An agency team member can explain the architecture, trade-offs and known limitations without reading a script.

11. Continuity brief — paste this into any future implementation chat

Project: NECTELIS Reusable Proof-of-Capability Systems. We are a three-member technical Upwork agency covering AI Apps & Integration, Web Development, AI & Machine Learning, and ERP/CRM Software. Our objective is to build **two honest, independently working, easily configurable demonstration systems** used repeatedly for proposals, Upwork portfolios, GitHub and short technical videos. We do **not** rebuild a custom proof per client and do **not** exaggerate what has been tested.

Proof 1, ClientFlow AI: configurable inbound lead → validate → duplicate event check → contact lookup/upsert → deterministic qualification → CRM pipeline → calendar availability/booking or alternative → team notification → audited result. Prefer n8n plus one simple FastAPI or Node adapter layer, PostgreSQL for state, synthetic data, and a clearly labeled mock mode; connect authorized GHL, Google Calendar or Vapi/Twilio test integrations only when access exists. Demonstrate success, duplicate input, unavailable booking and API failure; provide two industry profiles, README, tests, screenshots, short videos and a case study.

Proof 2, SyncGuard: one-way source API → normalized records → composite sync key + deterministic signature → mapping lookup → create/update/skip destination → only then persist successful state → bounded error retries or quarantine → auditable run summary. Mock source/destination APIs first; optional authorized Microsoft Graph/GHL connector later. Prove first create, unchanged skip, one changed-field update, per-recipient isolation, 429 handling, invalid record, partial-failure replay and clean counts.

Operating rules: simplicity first, no paid services unless necessary, explicit mock/live labels, no public credentials or customer data, no fake production claims, strong edge-case testing, reusable config/adapters, no unbounded features. Prioritize a small working ClientFlow proof for a currently active GHL job; expand after submission. Work step by step, explain each decision clearly, and never call a capability complete without executed evidence.

End of master blueprint. The first real action is **P0: check which test environments and existing authorized work are available**, then build the shortest verifiable ClientFlow success and duplicate-event paths before adding optional AI voice, UI polish or social integrations.